

Da ideia ao projeto **organizado**

Processo de software, organização do projeto e qualidade local.

Um processo simples para transformar uma ideia em projeto antes de gerar código.

2H BASE → 3H COM OPCIONAIS → 6 BLOCOS → PÚBLICO NÃO DEV

Agenda do dia

7 blocos · 120 minutos · duas práticas produzindo o PRD da dupla.

Bloco	Tema	Duração
1	Abertura + demonstração do vazamento	13 min
2	Processo, problema, escopo e não escopo	23 min
3	Prática: PRD do próprio projeto	15 min
4	Critérios de aceite e dividir em tarefas	18 min
5	Prática: completar o PRD	12 min
6	Estrutura, documentação e validação local	22 min
7	Versionar o PRD + primeiro contato com o agente	17 min
	Total	120 min

Agentes aceleram o código. O processo mantém o controle.

O que acontece antes do código é o que decide se o projeto se sustenta.

PONTO DE PARTIDA CAÓTICO

Sem processo

- Cada projeto nasce diferente do outro
- O agente preenche sozinho tecnologia, pastas, bibliotecas e arquitetura
- Funciona na demo, mas é difícil de continuar
- Depende demais da conversa original

FLUXO SOB CONTROLE

Com processo

- Um fluxo único e reconhecível
- Ideia → plano → template → tarefas → validação
- As decisões importantes continuam com a equipe
- O agente executa, mas não decide sozinho

Tudo o que não definimos vira uma decisão do agente

Um pedido curto carrega uma intenção — não define o produto.

PROMPT VAGO

"Crie um sistema de tarefas moderno."

- › React ou outra tecnologia?
- › Onde salvar os dados?
- › Login real ou simulado?
- › Quais telas e bibliotecas?
- › Como testar? Quando está pronto?

1

Decisões de negócio

- Quem usa, o que resolve, o que é obrigatório.

2

Decisões de escopo

- O que entra agora e o que fica para depois.

3

Decisões técnicas

- Estrutura, ferramentas permitidas e como validar.

Processo em cinco etapas

Uma etapa produz a entrada da próxima. O agente ajuda em todas — mas não pulamos para a quarta.

01

Definir

O problema, de forma que outra pessoa entenda.

02

Especificar

Funcionalidades e critérios de aceite.

03

Dividir

Quebrar o trabalho em tarefas menores.

04

Implementar

Um incremento por vez.

05

Validar

Antes de registrar como concluído.

Definir o problema

Uma boa definição responde quatro perguntas.

Quatro perguntas

- Para quem? — quem vai usar a aplicação
- Qual problema? — que dificuldade resolve
- Qual resultado? — o que a pessoa consegue fazer
- Qual limite? — o que não entra nesta versão

PREENCHIDO

Exemplo: TaskWeather

- Para quem organiza tarefas pessoais
- Registrar tarefas e ver o clima sem trocar de tela
- Acessar, criar tarefas e visualizar o clima
- Sem cadastro real, compartilhamento ou sincronização

Escopo e não escopo

Dizer não agora evita retrabalho depois.

Entra agora

- Entrada por e-mail, sem senha
- Dados separados por usuário
- Tarefas locais
- Clima atual

ETAPA FUTURA

Não entra agora

- Autenticação real
- Senha e recuperação de senha
- Cadastro
- Colaboração
- Banco de dados
- Previsão de 7 dias

Frases explícitas: “Não implemente...” · “Não altere...” · “Não instale...” · “Considere como etapa futura...”

Funcionalidade não é critério de aceite

Funcionalidade descreve uma capacidade. Critério descreve como verificá-la.

Funcionalidade: “Criar tarefa”

- Ainda deixa dúvidas: título vazio? quando aparece? persiste?

-  Rejeitar título vazio
-  Mostrar a tarefa criada
-  Manter a tarefa após recarregar

CRITÉRIOS OBSERVÁVEIS

Exemplo guiado: entrada por e-mail

- E-mail válido permite acesso
- E-mail vazio ou inválido mostra mensagem e não entra
- Atualizar a página mantém a sessão
- “Sair” retorna à tela de entrada
- As tarefas de um e-mail não aparecem para outro

Dividir para controlar

A unidade ideal é uma alteração pequena, com objetivo claro e validação própria.

UMA CAIXA GRANDE → CARTÕES MENORES

Criar TaskWeather

1. preparar projeto
2. entrada por e-mail
3. criar tarefas
4. concluir tarefas
5. consultar clima
6. revisar
7. documentar

REGRA PRÁTICA

Uma boa tarefa

- Tem um único objetivo
- Cabe em uma sessão de trabalho
- Altera um conjunto limitado de arquivos
- Possui critérios de aceite próprios

GitHub como memória do projeto

O projeto não pode existir apenas na conversa com o agente.

Repositório

- O projeto e seus arquivos.

Issue

- O trabalho a realizar.

Histórico

- Quem alterou o quê.

Pull request

- Revisão antes de incorporar.

Uma conversa com o agente pode ser perdida ou substituída. O repositório permanece.

Branch, commit e pull request

Não precisamos dominar Git avançado; precisamos de um fluxo consistente.

main → **feat/todo-list** → **commits** → **pull request** → **revisão** → **merge**

NOMES DE BRANCH

```
feat/login-email  
feat/todo-list  
feat/weather-widget
```

COMMITTS

```
feat: adiciona identificação por e-mail  
feat: cria e conclui tarefas  
fix: trata erro na consulta  
docs: atualiza execução
```

O fluxo

- main é a versão estável
- Cada funcionalidade nasce em uma branch separada
- Cada conjunto coerente de mudanças vira um commit
- Pronto e testado → abre pull request para revisar

Estrutura por funcionalidades

Primeiro procure a funcionalidade. Depois o tipo de arquivo dentro dela.

ORGANIZAÇÃO POR FEATURE

```
src/  
├── app/  
├── features/  
│   ├── auth/  
│   ├── todos/  
│   └── weather/  
└── shared/
```

MAIS FÁCIL PARA QUEM NÃO É DEV

Como ler

- Alterar o login? → auth
- Alterar tarefas? → todos
- Alterar clima? → weather
- shared: só o que é realmente reutilizado
- app: a montagem geral da aplicação

Documentação mínima

Um projeto organizado deve ser entendido sem depender de quem o criou.

README.md

- Objetivo e como executar.

AGENTS.md

- Regras gerais para agentes de código.

CLAUDE.md

- Orientações para o Claude Code.

docs/architecture.md

- Organização, responsabilidades e decisões.

Não precisam ser longos. Precisam estar corretos e atualizados.

Validação local

Sem CI ainda. O foco agora é validar localmente — cada comando responde a uma pergunta.

npm run dev

A aplicação abre?

npm run test

Os comportamentos
funcionam?

npm run lint

Há problemas de padrão?

npm run build

O projeto gera uma versão
final?

Não considere a tarefa concluída enquanto houver erro.

Quando uma tarefa está pronta?

O agente dizer “concluído” não encerra a tarefa. Esta lista será reutilizada em todos os projetos.

- Critérios atendidos
- Aplicação funciona
- Testes passam
- Lint passa

- Build passa
- Documentação atualizada
- Commit criado
- Revisão realizada

Atividade em duplas: transformar ideia em plano

“Crie uma página para registrar tarefas e mostrar o clima.” — 7 min para preencher seis campos.

Campos a preencher

- Problema
- Público
- Escopo
- Não escopo
- Critérios
- Tarefas

CORRIGIR SÓ ESCOPO E CRITÉRIOS

Resposta de referência

- Organizar tarefas e ver o clima em uma página
- Escopo: e-mail sem senha, dados por usuário, tarefas, persistência, clima
- Não escopo: cadastro, colaboração, banco, previsão estendida
- Critérios: ≥ 2 por funcionalidade

Três hábitos para levar

- Não começar pelo código
- Deixar explícitos escopo, não escopo e critérios
- Validar localmente antes de considerar uma mudança pronta